

# Intro to Supervised Learning

Albert Y. Kim

# Overview

- ▶ Applications of *unsupervised learning* to tend to be open-ended and prone to all sorts of subjective choices
  - ▶ Methods like scree plots of silhouette scores provide some objective guidance, but it's difficult to judge one analysis to be quantitatively better than another

# Overview

- ▶ Applications of *unsupervised learning* tend to be open-ended and prone to all sorts of subjective choices
  - ▶ Methods like scree plots of silhouette scores provide some objective guidance, but it's difficult to judge one analysis to be quantitatively better than another
- ▶ In contrast, *supervised learning* tends to be highly objective
  - ▶ We have a predetermined outcome  $y$  that we're aiming to predict, which allows us to quantify how accurate our predictions are

# Overview

- ▶ Applications of *unsupervised learning* tend to be open-ended and prone to all sorts of subjective choices
  - ▶ Methods like scree plots of silhouette scores provide some objective guidance, but it's difficult to judge one analysis to be quantitatively better than another
- ▶ In contrast, *supervised learning* tends to be highly objective
  - ▶ We have a predetermined outcome  $y$  that we're aiming to predict, which allows us to quantify how accurate our predictions are
- ▶ We'll focus on two types of supervised learning
  - ▶ **Classification** - predicting  $y$  the class or category of a data-point
  - ▶ **Regression** - predicting  $y$  a numerical characteristic of a data-point

# Supervised learning framework

- ▶ Consider data consisting of an  $n$ -dimensional vector of outcomes,  $\mathbf{y}$ , an  $n$  by  $p$  matrix of features,  $\mathbf{X}$
- ▶ Suppose the **true, usually unknowable** relationship between  $\mathbf{y}$  and  $\mathbf{X}$  is given by the following equation:

$$\mathbf{y} = f(\mathbf{X}) + \epsilon$$

where  $\epsilon$  is an  $n$ -dimensional vector of errors

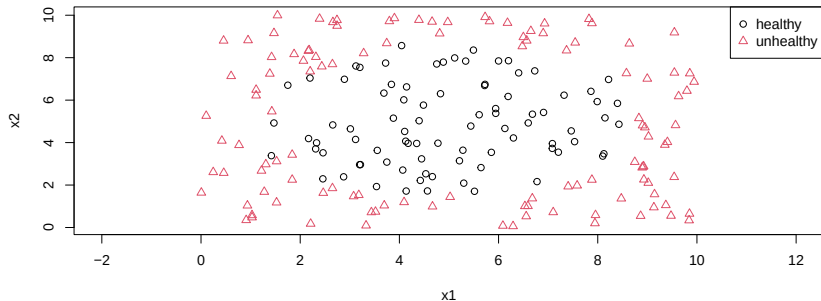
- ▶ We aim to accurately approximate  $f()$  via

$$\hat{\mathbf{y}} = \hat{f}(\mathbf{X})$$

- ▶ Note:
  - ▶ If  $\epsilon = \mathbf{0}$ , we may be able to perfectly approximate  $f()$
  - ▶ However,  $\epsilon \neq \mathbf{0}$  introduces *irreducible error*

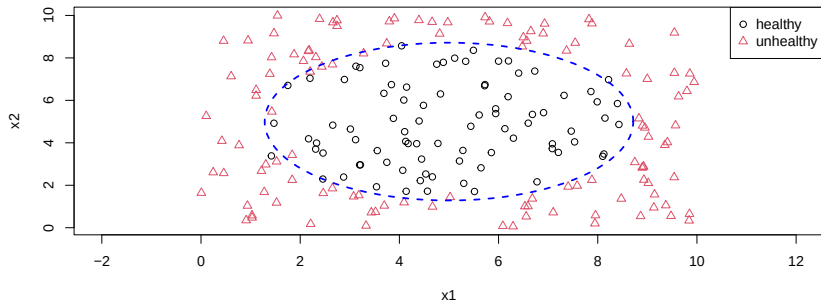
## Example

Consider two predictors,  $X_1$  and  $X_2$ , and a binary outcome  $Y$  of “healthy” or “unhealthy”. Can these predictors be used to accurately *classify* an observation?



## Example (cont)

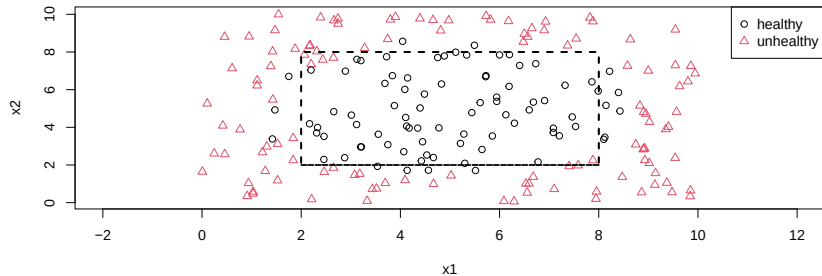
In this example, say the *true, typically unknown*  $f()$  is shown below (blue ellipse):



We can see that it's possible to use the data to learn a good approximation  $\hat{f}()$  of  $f()$ .

## Example (cont.)

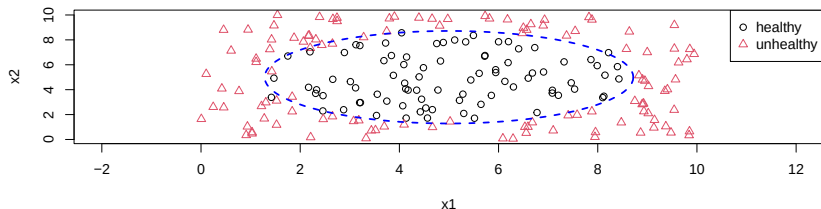
As a human, you might observe that healthy data-points tend to fall between 2 and 8 in both  $x_1$  and  $x_2$ , so you might propose the following *classification model*:



This simple model correctly classifies 178 of 200 data-points.

## Example (irreducible error)

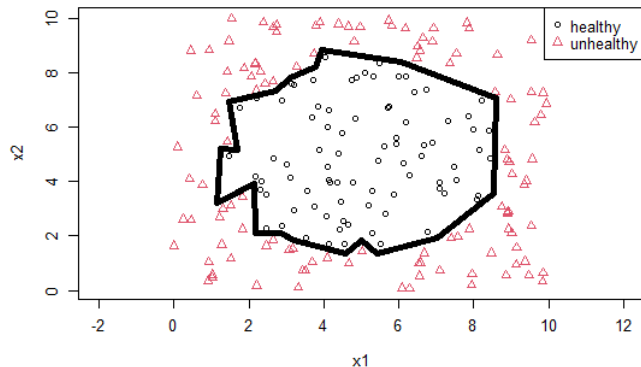
Let's revisit the true relationship between  $X_1$ ,  $X_2$ , and  $Y$ . Notice how some “healthy” data-points are outside the ellipse, and some “unhealthy” ones are inside it:



- ▶ The misclassification of these data-points reflects this scenario's **irreducible error**  $\epsilon$ , sometimes called “Bayes error”)
- ▶ Even the *best possible approximation*  $\hat{f}()$  of  $f()$  cannot perfectly classify every data-point

# Reducible Error

Achieving the best approximation  $\hat{f}()$  of  $f()$  amounts to minimizing *reducible error*. Consider the following classifier:



This classifier has a zero error rate.

# Training vs. Testing Splits

- ▶ We aren't really interested in the error rate for *observed examples*. Instead, we'd like to minimize reducible error on *new examples* that our model *hasn't seen yet*

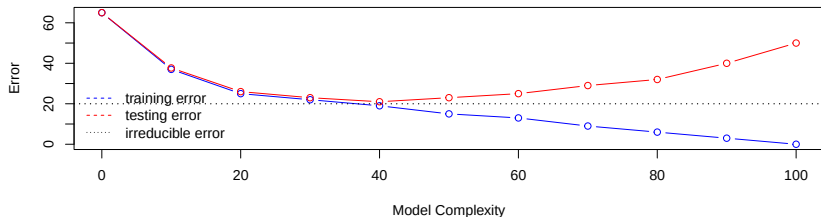
# Training vs. Testing Splits

- ▶ We aren't really interested in the error rate for *observed examples*. Instead, we'd like to minimize reducible error on *new examples* that our model *hasn't seen yet*
- ▶ Standard protocol is to split the available data into **training** and **testing** sets
  - ▶ The training set is used to *learn* a collection of rules
  - ▶ The testing set is used to *validate* how well these rules perform on unseen data

# Training, Testing, and Error

Consider a hypothetical example with an irreducible error of “20 units”

- ▶ Training error can be reduced by increasing model complexity (ie: learning more rules)
- ▶ Test error is bounded in probability by the irreducible error



# Bias vs. Variance

Reducible error can arise in one of two ways: **bias** or **variance**

- ▶ **Bias** is when a learner lacks the structural flexibility to detect aspects of the true relationship between the predictors and the outcome
- ▶ **Variance** is when a learner is overly sensitive to chance artifacts present in the data (ie: the manifestations of irreducible error)

Poor performance due to high bias is called *underfitting*, while poor performance due to high variance is called *overfitting*

# Bias vs. Variance

How would you compare the bias and variance of the following models (a rectangle vs. an n-dimensional polygon)?

